Spendly

Personal Expense Tracker and Analysis System

Knowledge Transfer (KT) Document

Version 1.0 · 9 October 2026

What is this document?

A simple guide to the whole project. It explains what every part does, where it lives, and how the parts talk to each other. You do not need to know the project before reading it.

Who should read it	What you will be able to do afterwards
New developers, testers, reviewers, and anyone who has to run or support the project.	Explain how it works, run it on your computer, find the right file for any change, and fix common problems.

Technology used
HTML · CSS · JavaScript (frontend) | Python · Flask (API) | Pandas (analysis) | Matplotlib (charts) | MongoDB and Compass (database)

Contents

1. What is this project? 4
2. The big picture 5
3. Project folder map 7
4. The frontend (what the user sees) 8
5. The middle layer: the Flask API 11
6. Validation and security 13
7. Storage: MongoDB and the JSON backup 15
8. Settings and categories (nothing is hardcoded) 17
9. Analysis with Pandas (analytics.py) 18
10. Charts with Matplotlib (charts.py) 19
11. Follow the data: four examples 20
12. How to run it (Windows) 21
13. Testing 22
14. Troubleshooting 22
15. How to make common changes 23
16. Known limits and ideas for the future 24
17. Glossary (simple words) 24
18. Quick reference card 25

How to use this document

If you want to…	Read
Understand the project in 10 minutes	Sections 1, 2 and 5
Run it on your computer	Section 12 (then 14 if something goes wrong)
Understand where the data is kept	Sections 7 and 8
Understand the numbers and charts	Sections 9 and 10
Change or extend the code	Sections 3, 4, 5 and 15
Look up a word you do not know	Section 17 (glossary)

1. What is this project?

1.1 In one paragraph

Spendly is a website that runs on your own computer. You use it to write down the money you spend each day, and then it shows you where your money goes. You can add, edit, search and delete expenses. It also draws charts, builds monthly reports, and gives short tips such as *"You spent 31% of this month's expenses on Bills."* Your data is saved in a database called MongoDB, and you can open the same data in a tool called MongoDB Compass.

1.2 What the app can do

- **Dashboard:** total spent, this month, average expense, largest expense, a trend chart, a category chart, recent expenses and tips.
- **Add Expense:** a form with date, category, amount and description. Bad input is rejected with a friendly message.
- **Expenses:** a table with search, filters (date, category, amount), sorting, pages, edit, delete and CSV export.
- **Analytics:** highest and lowest expense, most expensive category, daily / monthly / per-category averages.
- **Reports:** a monthly report, a month-to-month comparison and four charts drawn by Matplotlib.
- **Categories:** see how much you spent in each category.
- **Settings:** change your name, tagline, app name, currency and theme, and add, rename, recolor or delete categories.
- **Works on any screen:** desktop, tablet and phone, in light or dark mode.

1.3 An easy way to picture it: a restaurant

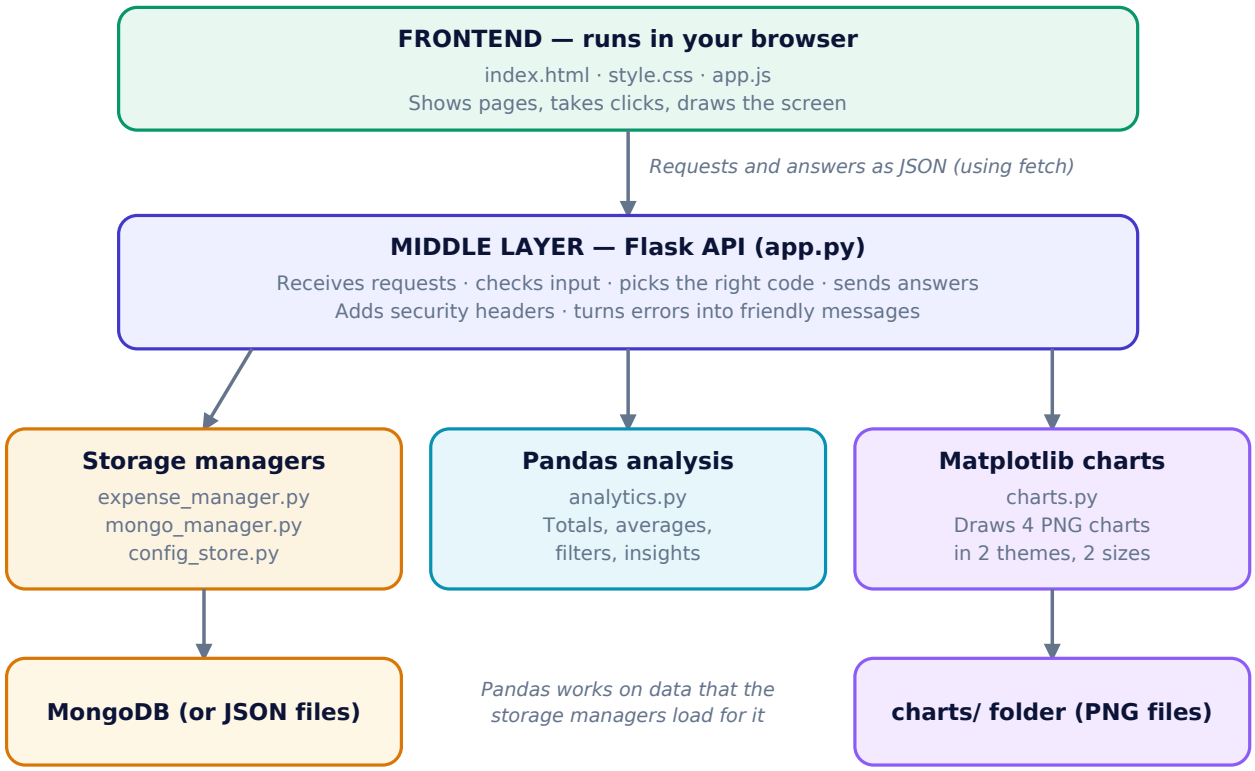
In a restaurant	In Spendly	Where it lives
The customer who orders	You, in your web browser	Your screen
The menu, tables and decoration	The **frontend**: pages, buttons, colors, forms	`frontend/` folder
The waiter who carries orders to the kitchen and brings food back	The **middle layer**: the Flask API. It takes your request, checks it, passes it on, and returns the answer	`backend/app.py`
The chef and the kitchen rules	The **logic**: validation rules, Pandas calculations, Matplotlib charts	`expense_manager.py`, `analytics.py`, `charts.py`
The storeroom	The **database**: MongoDB (or JSON files as a backup plan)	MongoDB database `spendly`
The shop's own settings (opening hours, menu items)	The **configuration**: currency, categories, name, theme	`config_store.py` and the `config` collection

1.4 What the project does not do (on purpose)

- **No login.** It is a single-user app. Anyone who can open the address sees the same data. Use it on your own computer only.
- **No cloud or sync.** The data stays in your own MongoDB. It does not sync between devices by itself.
- **No budgets or reminders yet.** The notification bell shows tips calculated when the page loads. It is not a push notification system.

2. The big picture

2.1 The layers



Read the picture from the top down. The browser never talks to the database directly. Every request goes through the Flask API. This keeps the rules in one place: if a rule is checked in the API, no screen can skip it.

2.2 Where is what used? (the master table)

Layer	Technology	Files	Its job
Frontend (user interface)	HTML, CSS, JavaScript. No framework.	`frontend/index.html` `css/style.css` `js/app.js`	Draws every page, form, table and SVG chart. Sends requests to the API. Shows toasts and errors.
Middle layer (API, “middleware”)	Python and Flask	`backend/app.py`	Receives requests, reads the input, calls the right code, builds the JSON answer, handles errors, adds security headers.
Validation (the gatekeeper)	Python	`expense_manager.py` (clean_expense) `config_store.py`	Decides if data is allowed. Fixes small things (extra spaces) and rejects bad data.
Storage (the memory)	MongoDB with pymongo, or JSON files	`mongo_manager.py` `expense_manager.py` `config_store.py`	Saves and reads expenses, settings and categories.
Analysis (the calculator)	Pandas and NumPy	`analytics.py`	Totals, averages, group-by, filters, sorting, monthly numbers, insights.
Charts (the painter)	Matplotlib	`charts.py`	Draws bar, pie, line and histogram charts as PNG pictures.
Demo data and tests	Python	`sample_data.py` `tests*.py`	Creates realistic sample expenses. Checks that everything works.
Database viewer	MongoDB Compass	Not part of the code	A desktop app to look at and edit the database by hand.

2.3 What does “middleware” mean in this project?

> **Simple meaning**
> **Middleware is the part in the middle** — between what the user sees (frontend) and where data is kept (database). In Spendly that middle part is the **Flask API** in `backend/app.py`. People also call it the backend or the API layer. They all mean the same place here.

Flask also uses the word in a smaller, technical way: small pieces of code that run on *every* request. Spendly has these (all in `app.py`):

What runs on every request	What it does	Why it matters
`MAX_CONTENT_LENGTH` = 64 KB	Rejects very large request bodies with status 413.	Stops someone from sending huge data.
`body()` helper	Makes sure the request body is a JSON object.	Bad or empty bodies get a clear 400 error.
`context()` (per request)	Reads the settings and category list once and reuses them during the request.	Fewer database reads and the same rules all through one request.

What runs on every request	What it does	Why it matters
`@app.after_request` hook	Adds CORS headers for local addresses, `X-Content-Type-Options: nosniff`, and `Cache-Control: no-store` on API answers.	Lets the page work from localhost or a file, and stops the browser from keeping old API answers.
Error handlers	Turn every error (bad input, not found, database down, crash) into a clear JSON message.	The user never sees a Python error or a stack trace.

2.4 Technology list

Technology	Version rule	Used for
Python	3.11 or newer	All backend code. Pandas 3 needs Python 3.11+.
Flask	3.1 or newer (below 4)	The web server and API. Also serves the frontend files.
Pandas (and NumPy)	3.0 or newer (below 4)	All analysis: totals, groups, filters, sorting, averages.
Matplotlib	3.10 or newer (below 4)	The PNG charts on the Reports page.
pymongo	4.8 or newer (below 5)	Lets Python talk to MongoDB.
MongoDB Community Server	Any recent version	The database. Runs on port 27017.
MongoDB Compass	Any recent version	A viewer for the database. Not needed to run the app.
mongomock (tests only)	4.1 or newer	A fake in-memory MongoDB so the tests need no server.
Browser	A modern one (Chrome, Edge, Firefox, Safari)	Runs the frontend. Needs CSS variables and container queries.

3. Project folder map

```
expense-tracker/
├── frontend/
│   ├── index.html           the page skeleton: sidebar, top bar, page sections, pop-up dialogs
│   ├── css/style.css        all colors, layout, dark mode and responsive rules
│   └── js/app.js            all the browser logic (about 540 long lines)
├── backend/
│   ├── app.py               Flask app: all API routes, error handling, security headers
│   ├── expense_manager.py   validation rules + JSON-file storage
│   ├── mongo_manager.py     MongoDB storage (same methods as the JSON one)
│   ├── config_store.py      settings, currency and categories (stored in the database)
│   ├── analytics.py         Pandas calculations
│   ├── charts.py            Matplotlib charts
│   ├── sample_data.py       demo expenses for the first run
│   ├── tests.py             API + JSON storage tests
│   ├── tests_mongo.py       MongoDB storage tests
│   ├── tests_config.py      settings + categories tests
│   └── data/                used only in JSON mode (expenses.json, config.json)
├── charts/                  the PNG charts are saved here
├── requirements.txt         packages needed to run
├── requirements-dev.txt     adds mongomock for the tests
└── README.md
```

Tip: if you are lost, ask “is this about what the user sees, or about data?” Screen problems are in `frontend/`. Data, rules and numbers are in `backend/`.

4. The frontend (what the user sees)

4.1 What it is

The frontend is a **single-page app**. That means the browser loads one HTML page, and JavaScript swaps the content when you click a menu item. There is **no framework** (no React or Angular), only plain HTML, CSS and JavaScript. This keeps it small and easy to read.

File	Think of it as	What is inside
`index.html`	The skeleton	The sidebar, top bar, one empty section per page, the pop-up dialogs (edit expense, category, confirm), the toast area, the mobile bottom bar, and the icon set.
`css/style.css`	The skin	Colors as CSS variables (light and dark), layout, cards, tables, animations, and all the responsive rules.
`js/app.js`	The brain	Talks to the API, draws every page, validates forms, runs filters, shows toasts, and applies your settings.

4.2 The seven pages

Page	What the user sees	API calls it makes
Dashboard	4 summary cards, trend chart, category donut, recent expenses, tips	`GET /analytics/summary`, `/analytics/categories`, `/analytics/monthly`, `/expenses?per_page=6`
Add Expense	The form, quick category chips, a “recently added” list, a success animation	`POST /expenses`, `GET /expenses?per_page=5`
Expenses	Search and filter bar, table (cards on phones), edit and delete, pages, CSV export	`GET /expenses` with filters, `GET/PUT/DELETE /expenses/<id>`, `GET /export`
Analytics	Totals, highest and lowest, top and most frequent category, averages, category bars	`GET /analytics/summary`, `/analytics/categories`
Reports	Pick a month and a month to compare, plus four Matplotlib chart pictures	`GET /analytics/report`, `/analytics/charts`
Categories	One card per category with total, share, count and average	`GET /analytics/categories`
Settings	Profile, currency, theme, export, sample data, delete all, category manager	`GET /config`, `PUT /settings`, `/categories` (add, edit, delete)

4.3 How `app.js` is organised

Section (top to bottom)	Main names inside	Job
Helpers and state	`CATS`, `CUR`, `fmt`, `esc`, `S`	Number formatting using the chosen currency, safe HTML escaping, and one object `S` that remembers the current page and filters.
API client	`call`, `api`, `downloadFrom`	The only place that uses `fetch`. Turns server errors into friendly messages.
Validation	`validate`	Checks the form before sending (the server checks again).
SVG charts	`lineChart`, `bars`, `donut`	Hand-drawn charts that follow the theme and shrink on phones.
Shared UI pieces	`statCard`, `toast`, `confirmBox`, `countUp`, `failure`	Reusable cards, messages, dialogs, counting animation and the “try again” card.
Forms	`initAdd`, `openEdit`, `askDelete`	Add form, edit dialog, delete confirmation.
Filters	`filterQS`, `initFilters`	Builds the query string from the search, date, category, amount and sort controls.
Pages	`R.dashboard` … `R.settings`	One function per page. Each one fetches data and builds the HTML.
Settings and categories	`openCat`, `saveCat`, `deleteCat`	The category manager dialogs.
Shell	`route`, `render`, `applyConfig`, `loadConfig`, `init`	Navigation, theme, sidebar, notifications, and start-up.

4.4 How a page appears on the screen

1. You click “Expenses” in the menu. The address changes to `#expenses` (called a hash route).
2. The browser fires a `hashchange` event, and `route()` runs. It hides all page sections and shows `#p-expenses`.
3. `render()` calls the matching page function, `R.expenses()`.
4. That function asks the API for data using `api.list(...)`. While waiting, a grey “skeleton” placeholder is shown.
5. When the data arrives, the function builds an HTML string and puts it in the page with `innerHTML`. Every piece of user text goes through `esc()` first.
6. If the server cannot be reached, `failure()` shows a “Can’t reach the server” card with a **Try again** button.

4.5 Two kinds of charts

Where	How they are made	Why
Dashboard and Analytics	Drawn in the browser as SVG by `lineChart`, `bars`, `donut`. The numbers come from Pandas.	Interactive (hover tips), follows light and dark mode, sharp on any screen.
Reports	PNG pictures drawn by Matplotlib on the server and shown with an `img` tag.	Matches the project brief, can be reused in reports, and looks the same everywhere.

4.6 Look and feel

- **Design tokens:** colors, shadows and sizes are CSS variables at the top of `style.css`. Change them in one place to re-theme the whole app.
- **Dark mode:** setting `data-theme="dark"` on the page swaps the variable values. The choice is saved with your settings in the database, and a copy is kept in the browser so the page does not flash white on load.
- **Animations:** cards slide in, numbers count up, line charts draw themselves. Everything switches off for people who ask their device for reduced motion.

4.7 Responsive design (desktop, tablet, phone)

Screen width	What changes
Above 1600 px	The content area grows a little wider.
1180 px and below	The date chip in the top bar is hidden to save room.
1060 px and below	The summary cards go from 4 per row to 2 per row.
1000 px and below	Two-column sections (chart + side panel) become one column.
900 px and below (tablet)	The sidebar becomes a slide-in drawer. A **bottom navigation bar** appears (Home, Expenses, Add, Analytics, More). The top bar wraps into rows, and the search box gets its own row.
620 px and below (phone)	Tables turn into cards. Filters form a 2-column grid. Dialogs slide up as bottom sheets. Inputs use 16 px text so iPhones do not zoom. Charts are redrawn smaller (SVG) or requested in a compact size (Matplotlib).
380 px and below	Summary cards go to one per row.
Touch screens	Buttons get bigger tap areas. Notch and home-bar spacing is handled with safe-area insets.

Good to know
The summary card numbers use **CSS container queries**. The text shrinks to fit the card, so a long amount like ₹12,34,567 never overflows on a narrow phone.

4.8 Safety inside the browser

- Every value shown on screen is escaped with `esc()`, so a description like `<script>` is shown as text and never runs.
- The form validates first so users get quick feedback. The server still re-checks everything.
- Buttons are disabled while saving, so a double click cannot create two expenses.

5. The middle layer: the Flask API

5.1 What `app.py` does

- `create_app()`**:** builds the Flask app. It picks the storage, loads the config store, and registers all routes. Tests create their own copy with temporary data.
- `build_manager()`**:** chooses MongoDB or the JSON file (see section 7).
- **Routes:** small functions. Each one reads the request, calls one storage or analytics function, and returns JSON. They hold almost no business logic.
- **Serves the frontend:** the same server sends `index.html`, the CSS and the JavaScript, so you only start one program.

5.2 All endpoints

Method	Address	What it does
GET	`/api/expenses`	List expenses. Filters: `q, category, range, from, to, min, max, sort`. Paging: `page, per_page` (default 10, up to 100).
POST	`/api/expenses`	Add an expense. Returns 201.
GET / PUT / DELETE	`/api/expenses/<id>`	Read, replace or delete one expense. 404 if it does not exist.
DELETE	`/api/expenses?confirm=true`	Delete every expense. Needs `confirm=true` as a safety catch.
POST	`/api/expenses/sample`	Replace all data with sample data. Categories you removed are mapped onto ones you still have.
GET	`/api/analytics/summary`	Totals, averages, highest, lowest, top categories, this month vs last month, and the tips.
GET	`/api/analytics/categories`	Totals per category. Optional `month=YYYY-MM`.
GET	`/api/analytics/monthly`	Monthly totals. `months=6` by default (1 to 24).
GET	`/api/analytics/report`	Monthly report and comparison. `month` and `compare` are optional.
GET	`/api/analytics/charts`	Makes (or reuses) the four PNG charts. `theme=light|dark`, `size=wide|compact`.
GET	`/charts/<file>`	Sends a chart picture.
GET	`/api/export`	`format=csv|json`. Accepts the same filters as the list.
GET	`/api/config`	Settings, categories, current currency, list of currencies, storage info.
PUT	`/api/settings`	Save any of: app name, display name, tagline, currency, theme, sidebar state.
GET / POST	`/api/categories`	List categories with how many expenses each has / add a category.
PUT / DELETE	`/api/categories/<name>`	Edit or rename / delete. Delete needs `reassign_to` when the category has expenses.
GET	`/api/health`	Is the server up, which storage is used, how many expenses (503 if the database is down).

5.3 Status codes you will see

Code	Meaning	Example
200	OK	A list, a summary, a saved setting.
201	Created	A new expense or category was added.
400	Bad request	Amount is `abc`, date is in the future, a category name already exists. The answer includes `fields` saying which field is wrong.
404	Not found	Expense id 999999 does not exist.
405	Method not allowed	Using PATCH on `/api/expenses`.
413	Too large	A request body over 64 KB.
500	Server problem	The database stopped. The message is friendly. The technical details are written only to the server log.
503	Unavailable	The health check when the database cannot be reached.

5.4 Walk-through: what happens when you click “Add expense”

1. **Browser:** the form submit handler runs `validate()`. If something is wrong, the message is shown under the field and nothing is sent.
2. **Browser:** `api.add()` sends `POST /api/expenses` with the data as JSON. The Save button is disabled.
3. **Flask:** before the route runs, the 64 KB limit is checked. The route calls `body()` to be sure it received a JSON object.
4. **Storage manager:** `manager.add(data)` calls `clean_expense()` with the *current* category list from the config store. It checks the date, category, amount and description, and tidies the text.
5. **MongoDB:** the next id is taken from the `counters` collection, and the document is inserted into `expenses`. (In JSON mode the file is rewritten safely instead.)
6. **Flask:** returns status 201 with the saved expense. The `after_request` hook adds the security headers.
7. **Browser:** shows the success animation and a toast, clears the form, and refreshes the data on screen. If the server had said 400, the field messages would be shown instead.

6. Validation and security

6.1 Rules for an expense

Field	Allowed	Not allowed
Date	A real date, written `YYYY-MM-DD`, from 2000-01-01 up to today.	Future dates, `2026-02-31`, `05-10-2026`, empty.
Category	Any of your categories. Capital letters do not matter (`food` becomes Food).	A category that does not exist.
Amount	A number above 0, up to 10,000,000, with at most 2 decimals. Text such as `500` is accepted.	`abc`, `-500`, `0`, `₹₹500`, `5.555`, true/false, empty, infinity.
Description	1 to 80 characters. Extra spaces are removed and hidden control characters are stripped.	Empty, only spaces, longer than 80 characters, not text.

6.2 Rules for settings and categories

Item	Rule
Display name / tagline / app name	Not empty. At most 30 / 30 / 24 characters.
Currency	One of INR, USD, EUR, GBP, JPY.
Theme	light or dark.
Category name	1 to 20 characters: letters, numbers, spaces, &, -, ' and _. No two names alike (capitals ignored).
Category emoji	One emoji or symbol (up to 8 characters). Empty gets a default tag icon.
Category color	A hex color like `#10b981`.
Number of categories	At least 1 and at most 24.

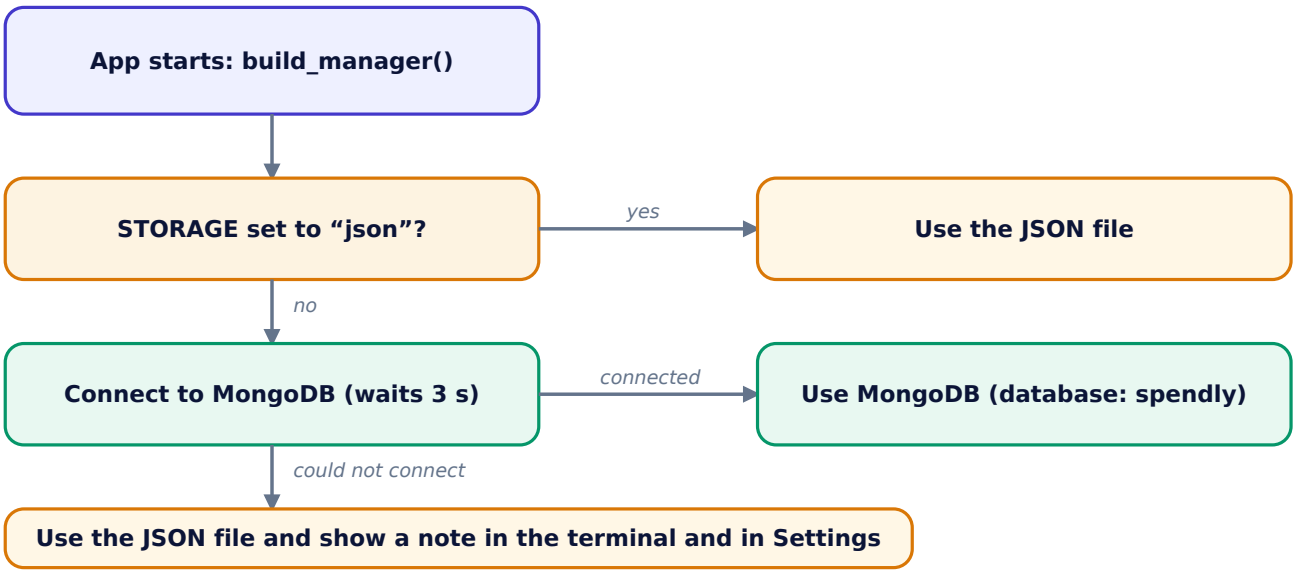
6.3 Security measures in place

- **The server is the boss.** Browser checks are only for comfort. Every write is validated again in Python.
- **No stack traces.** A crash returns a short friendly message. Details go to the server log only.
- **Safe display.** The frontend escapes all user text (protection against script injection).
- **CSV safety.** In exports, a description starting with `=`, `+`, `-` or `@` gets a leading apostrophe, so Excel cannot run it as a formula.
- **Safe file serving.** Charts are sent with Flask's `send_from_directory`, which blocks paths like `../app.py`.
- **Limited CORS.** Only `localhost`, `127.0.0.1` and local files (origin `null`) may call the API from a browser.
- **Hidden passwords.** If your MongoDB address has a username and password, the screen shows `***` instead.
- **Safe saving.** JSON files are written to a temporary file first, then swapped in, so a crash cannot leave half a file.

> **Important limit**
> There is **no login**. The app listens only on `127.0.0.1` (your own computer). Do not publish it on the internet as it is. If you need that, add user accounts first.

7. Storage: MongoDB and the JSON backup

7.1 How the app chooses where to save



MongoDB is the default. If it is not running, the app still works with JSON files and tells you clearly. Both storages offer the same methods, so the rest of the code does not care which one is used.

7.2 What is stored in MongoDB

Database name: `spendly`. Open it in Compass at `mongodb://localhost:27017`.

Collection	What is inside	Who changes it
`expenses`	One document per expense.	The app (and you, in Compass).
`config`	One document (`_id: "app"`) with your settings and category list.	The Settings page (and you, in Compass).
`counters`	The last used expense number, so ids are never reused.	The app only. Do not edit.
`meta`	A note that first-run setup is done, so “Delete all” is not undone when you restart.	The app only. Do not edit.

```
// one document in "expenses"
{ "_id": ObjectId("..."),  "id": 12,  "date": "2026-10-05",
  "category": "Food",  "amount": 450,  "description": "Lunch" }

// the one document in "config"
{ "_id": "app",
  "settings":   { "app_name": "Spendly", "display_name": "Subham", "tagline": "Personal plan",
                  "currency": "INR", "theme": "light", "sidebar_collapsed": false },
  "categories": [ { "name": "Food", "emoji": "□", "color": "#f59e0b" }, ... ] }
```

Note: `_id` is MongoDB's own key. `id` is the simple number the app uses in its web addresses (like `/api/expenses/12`). Both exist on purpose.

7.3 First run

1. The app connects and creates the indexes (a unique one on `id`, and one on `date`).
2. If the database has never been set up and the `expenses` collection is empty: it imports `backend/data/expenses.json` if that file has data. Otherwise it adds 85 sample expenses.
3. It then writes the `initialised` flag in `meta`. After that, the app never adds sample data by itself again.
4. The config store writes the default settings and the 8 default categories if none exist.

7.4 Editing in Compass is safe

You can add or change documents by hand. The app re-checks every document when it reads it:

- A document **without an** `id`, or with a duplicate id, gets a fresh id automatically.
- A date typed as a real date (BSON Date), or an amount typed as Decimal128, is understood.
- A category written in a different case (`food`) is matched to the right one.
- A document with invalid data (bad date, negative amount, unknown category) is **skipped, never deleted**, and a warning is written to the log.
- Changes made in Compass also refresh the chart pictures, because the app notices that the data changed.

7.5 JSON backup mode

- `backend/data/expenses.json` holds the expenses. `config.json` in the same folder holds the settings and categories.
- Saving writes a temporary file and then swaps it in (an *atomic* write).
- If `expenses.json` is damaged, the app renames it to `expenses.corrupt-<time>.json`, starts empty, and logs a warning. A damaged `config.json` falls back to defaults.
- The first time MongoDB is used after a JSON period, the JSON expenses are imported.

7.6 Environment variables

Variable	Default	Meaning
`STORAGE`	`mongo`	`mongo` or `json`
`MONGODB_URI`	`mongodb://localhost:27017`	Any MongoDB address, including Atlas (`mongodb+srv://...`).
`MONGODB_DB`	`spendly`	Database name.
`MONGODB_COLLECTION`	`expenses`	Expenses collection name.
`EXPENSE_DEBUG`	not set	Set to `1` to run Flask in debug mode. Use only while developing.

8. Settings and categories (nothing is hardcoded)

Everything you may want to change lives in the database and is edited in the **Settings** page. The code is in `backend/config_store.py`.

What	Where it is stored	Used by
App name, display name, tagline	`config.settings`	Sidebar, greeting, page title.
Currency (INR, USD, EUR, GBP, JPY)	`config.settings`	Every amount on screen, the Matplotlib charts, and the tip sentences.
Theme and sidebar state	`config.settings`	Applied when the app starts, so it follows your data.
Categories (name, emoji, color)	`config.categories`	Validation, filters, forms, Pandas, SVG and Matplotlib colors.

8.1 How it works

- `ConfigStore` reads the config document, **checks every value again**, and replaces anything invalid with a safe default. A typing mistake in Compass cannot break the app.
- `ConfigStore.context()` gives the API one ready-to-use bundle: category names, emoji, colors and the currency (symbol and locale).
- The storage manager asks the config store for the category list each time it validates or reads an expense: `manager.category_names = config.names`. This is why a new category works immediately.
- **Currency formatting** happens in two places: JavaScript `toLocaleString` in the browser, and `money_text()` in Python. For INR the grouping is 12,34,567 (lakh, crore). Other currencies use 1,234,567.

8.2 What happens when you change a category

Action	Steps
Add	Check name, emoji, color. Name must be new. Save in the config document. It shows up in every dropdown at once.
Edit color or emoji	Save in the config document. Nothing else changes.
Rename	Save the new name. Then `manager.rename_category(old, new)` moves every expense to the new name (in MongoDB one `update_many`). If that second step fails, the rename is undone so nothing is lost.
Delete (unused)	Remove it from the config document.
Delete (has expenses)	The API refuses until you choose `reassign_to`. The Settings page shows a dropdown for that. Expenses are moved first, then the category is removed.
Delete the last one	Refused. At least one category must always exist.

Chart pictures stay fresh
When settings or categories change, the config store changes its version number. The chart code compares that number with the age of each PNG and redraws only what is old. So a new currency or color shows on the Reports page right away.

9. Analysis with Pandas (`analytics.py`)

A **DataFrame** is like an Excel sheet inside Python: rows and columns that you can sort, filter and add up with one line. On every analytics request the app reads all expenses, puts them into a DataFrame with `to_df()`, and lets Pandas do the work. The columns are `id, date, category, amount, description` plus a `month` column like `2026-10`.

Function	Question it answers	How Pandas does it
`apply_filters`	Which expenses match the search, filters and sort?	True/False masks combined with `&`, `str.contains` for search, `sort_values` (stable sort).
`categories`	Total, count, average and share for each category	`groupby("category")` then `agg`. `reindex` keeps categories with zero spending in the list.
`monthly`	Spending for each of the last N months	`groupby("month")`. Missing months are filled with 0.
`monthly_series`	Every month from first to last expense (line chart)	`period_range` from the first to the last month.
`histogram`	How are amounts spread out?	NumPy `histogram` with 8 bins.
`summary`	All dashboard and analytics numbers	`sum`, `mean`, `idxmax` and `idxmin` (highest and lowest), `nunique` (months), date span.
`insights`	The tip sentences	Compares this month with the same days last month.
`report`	One month report and a comparison	Uses `categories` for both months, then percent change per category.

9.1 Formulas in plain words

Number	Calculation
Average expense	Total spent ÷ number of expenses.
Daily average	Total spent ÷ number of days from the first to the last expense (both days counted).
Monthly average	Total spent ÷ number of months that have at least one expense.
Category share	Category total ÷ total of all categories × 100.
Change in percent	(new − old) ÷ old × 100. Shown as empty when old is 0.
This month vs last month	Compares the same days. On 9 October it compares 1–9 October with 1–9 September. This is fair early in the month; comparing a few days against a full month would always look like a huge jump.

9.2 The tip sentences (insights)

Tip	Shown when
“You spent X% of this month’s expenses on Category.”	There is at least one expense this month.
“Category is your second-highest spending category this month.”	Two or more categories have spending this month.
“Your spending increased/decreased by X% compared with the same days last month.”	There are expenses in both periods.
“Your average expense rose/fell from A to B this month.”	There are expenses in both periods.
“Add a few expenses this month to see spending insights.”	There is nothing to compare yet.

The wording is a template. The numbers, category names and currency inside are calculated. The red dot on the bell appears when any tip is a warning or good news.

10. Charts with Matplotlib (`charts.py`)

Chart	Shows	Data comes from
Bar chart	Total spent in each category	`analytics.categories`
Donut (pie) chart	Percentage share of each category, total in the middle	`analytics.categories`
Line chart	Spending month by month over time	`analytics.monthly_series`
Histogram	How many expenses fall in each amount range	`analytics.histogram`

10.1 How it works

- The **Agg backend** is used, so charts are drawn in memory without opening any window. This is required on a server.
- Two **themes** (light and dark) with matching colors, so the pictures fit the app.
- Two **sizes**: `wide` (8 × 4.4 inches) for desktop, and `compact` (taller, bigger text) for phones. The browser asks for the right one.
- Category colors and the currency symbol come from your settings, so they always match the app.
- **Caching:** pictures are saved in `charts/` as `bar_light.png`, `bar_dark_compact.png` and so on. A picture is redrawn only if it is older than the latest data or settings change. The URL ends with `?v=<version>` so the browser also fetches the new one.
- A **lock** makes sure two requests never draw at the same moment, because Matplotlib is not fully thread-safe.
- With no data, each chart shows “No expenses yet” instead of failing.

Chart drawing is kept separate from the business logic on purpose: `charts.py` only draws what `analytics.py` calculates.

11. Follow the data: four examples

Example A — Opening the dashboard

1. The browser loads `index.html`, then `app.js`. `init()` calls `GET /api/config` and applies your name, theme, currency and categories.
2. `route()` sees `#dashboard` and calls `R.dashboard()`, which sends 4 requests at the same time (summary, categories, monthly, recent).
3. Flask builds one Pandas DataFrame per request from the stored expenses and calculates the answers.
4. The page draws the cards (numbers count up), the SVG charts, the recent list and the tips. The bell is updated from the same summary.

Example B — Renaming a category from Food to Meals

1. Settings → pencil icon → type “Meals” → Save. The browser sends `PUT /api/categories/Food`.
2. The config store checks the name and saves it. Then `rename_category` moves all Food expenses to Meals.
3. The browser calls `loadConfig()`, which refreshes every dropdown, filter and color.
4. The next time you open Reports, the chart pictures are redrawn because the config version changed.

Example C — Changing currency to USD

1. Settings → Currency → US Dollar. The browser sends `PUT /api/settings` with `{"currency": "USD"}`.
2. The answer contains the new currency. `applyConfig()` sets `CUR` to `$` and `en-US`.
3. All pages use `fmt()` and `money()`, which read `CUR`, so every number now shows $.
4. Python uses `money_text()` for tips and charts, so those switch too.

Example D — Editing a document directly in Compass

1. You change `amount` of one document in Compass.
2. On the next request, the storage manager reads all documents, re-checks each one, and notices that the data changed.
3. The change is on screen after you refresh the page, and the chart pictures are redrawn on the next visit to Reports.

12. How to run it (Windows)

12.1 One-time setup

1. Install **Python 3.11 or newer** (tick “Add Python to PATH”).
2. Install **MongoDB Community Server** (choose Complete and keep “Install MongoD as a Service”). Installing **MongoDB Compass** is optional.
3. Open PowerShell in the project folder and run:

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

12.2 Start the app

```
cd backend
python app.py
```

The terminal shows the address and which storage is used, for example:

```
Spendly is running → http://127.0.0.1:5000
Storage: MongoDB → spendly.expenses (mongodb://localhost:27017)
```

Open **http://127.0.0.1:5000** in the browser. If the Storage line says “JSON file”, MongoDB is not running. Start the *MongoDB Server* service (Win + R → `services.msc`), then restart the app.

12.3 See the data in Compass

1. Open Compass → **Add new connection** → paste `mongodb://localhost:27017` → **Connect**.
2. Open the database `spendly` and the collection `expenses`. The Settings page of the app has a “Copy connection string” button.

12.4 Next time

```
cd expense-tracker
.venv\Scripts\activate
cd backend
python app.py
```

> **macOS and Linux**
> Use `source .venv/bin/activate` instead of the Windows activate line. Start MongoDB with `brew services start mongodb-community` (macOS) or `sudo systemctl start mongod` (Linux).

13. Testing

File	Checks	What it covers
`tests.py`	94	Every API endpoint on JSON storage: add, read, edit, delete, filters, sorting, search, bad input, wrong IDs, empty data, damaged and missing files, chart files, CSV export, CORS.
`tests_mongo.py`	37	MongoDB storage: first-run import, ids never reused, hand-typed Compass documents repaired, database going down mid-request (friendly 500), fallback to JSON.
`tests_config.py`	80	Settings, currency, categories (add, edit, rename, delete, protection of the last one), stored bad config, on both JSON and MongoDB.

Total: 211 automated checks. To run them:

```
pip install -r requirements-dev.txt
cd backend
python tests.py
python tests_mongo.py
python tests_config.py
```

Each one ends with a line like `94 passed, 0 failed`. The tests use temporary data, so your real expenses are never touched.

What is checked by hand, not by tests

- **How it looks** (layout, colors, animation) on real screens. Check desktop, a tablet width and a phone width (about 375 px) before a release.
- **A real MongoDB server.** The automated tests use `mongomock`, a fake in-memory MongoDB. The app has also been run by hand against a local MongoDB.
- **Real browsers.** The page logic was also run in a simulated browser (jsdom) against the real server, but not in every real browser.

14. Troubleshooting

What you see	Likely cause	What to do
Compass says `ECONNREFUSED 127.0.0.1:27017`	The MongoDB **server** is not running or not installed. Compass is only a viewer.	Install MongoDB Community Server. Start the “MongoDB Server” service in `services.msc`. Check with `Test-NetConnection localhost -Port 27017`.
Terminal says “Using the JSON file instead”; Settings says Storage: JSON file	MongoDB was not reachable when the app started.	Start MongoDB, then stop the app (Ctrl + C) and run `python app.py` again.
Page shows “Can’t reach the server”	The backend is not running, or it stopped.	Run `python app.py` in the `backend` folder, then click Try again.
“Unable to reach the database” while using the app	MongoDB stopped after the app started.	Start MongoDB again. No restart of the app is needed.
`ModuleNotFoundError`	The virtual environment is not active, or packages are missing.	Run `.venv\Scripts\activate` and `pip install -r requirements.txt`.

What you see	Likely cause	What to do
Port 5000 already in use	Another program uses it (macOS AirPlay is common).	Change `port=5000` at the bottom of `backend/app.py`.
Old look or old behaviour after an update	The browser kept the old JavaScript or CSS.	Hard refresh with Ctrl + Shift + R.
Charts look old or are missing	Cached pictures.	Delete the PNG files in `charts/`. They are redrawn automatically.
A document added in Compass does not appear	It failed validation (bad date, unknown category, amount ≤ 0) and was skipped.	Look at the terminal for a “Skipping invalid document” warning, then fix the document.
Fonts look different	Plus Jakarta Sans loads from Google Fonts. No internet means the system font is used.	Nothing is broken. It looks the same once online.

15. How to make common changes

I want to…	Change this	Notes
Change colors or the look	`frontend/css/style.css` (variables at the top)	Dark mode values are in the `[data-theme=dark]` block.
Add a currency	`CURRENCIES` in `backend/config_store.py`	Give it a symbol, locale and name. It appears in Settings. Check the symbol exists in the chart font.
Change the default categories	`DEFAULT_CATEGORIES` in `config_store.py`	Only affects a brand-new database. Existing users change theirs in Settings.
Change how many rows per page	`PER` in `app.js`	The API allows up to 100.
Add a new page	1) a `<section id="p-name">` in `index.html`, 2) a menu link, 3) `R.name = async () => {...}` and its title in `TITLES` in `app.js`	The router finds it automatically.
Add a new API endpoint	A route in `app.py`, and (for numbers) a function in `analytics.py`	Keep the route short. Put the logic in the other files.
Add a new tip (insight)	`insights()` in `analytics.py`	Return `icon, tone, text`.
Add a chart to Reports	A drawing function and an entry in `CHARTS` in `charts.py`	The frontend shows every chart the API returns.
Save somewhere else	Set `MONGODB_URI` (for example Atlas) or `STORAGE=json`	No code change.

Example: adding a new expense field, “payment method”

1. **Validation:** in `clean_expense()` (`expense_manager.py`) read, check and add the new field. Add it in `coerce_record()` so stored data is read the same way.
2. **Pandas:** add the name to `COLUMNS` and `records()` in `analytics.py`. If you want charts by payment method, add a `groupby` function.

3. **Storage:** MongoDB and JSON need no change, because they store whatever the validation returns.
4. **Frontend:** add an input in `fieldsHTML()`, read it in `readForm()`, send it in `validate()`, and add a column in `R.expenses`.
5. **Tests:** add a few checks in `tests.py` for good and bad values, then run all three test files.

16. Known limits and ideas for the future

Known limits

- Single user, no login, local use only.
- The app reads all expenses for each analytics request. This is perfectly fast for personal use (thousands of rows), but a very large dataset would need database-side calculation.
- The tip sentences use fixed wording. Only the numbers and names are dynamic.
- Five currencies are included. Amounts are not converted between currencies (changing currency only changes the symbol and format).
- Dates are shown in the format day-month-year (for example 05 Oct 2026).

Good next steps

- **Monthly budgets** per category with alerts at 80% and 100%.
- **User accounts** (login) so more than one person can use it safely.
- **Recurring expenses** (rent, subscriptions) added automatically.
- **PDF export** of the monthly report.
- **Installable app (PWA)** so it can be added to a phone’s home screen.

17. Glossary (simple words)

Word	Meaning
API	A set of web addresses the frontend can call to ask for data or to save data. Like a waiter taking orders.
REST	A common style for APIs: use GET to read, POST to create, PUT to change, DELETE to remove.
Endpoint	One address of the API, such as `/api/expenses`.
JSON	A simple text format for data, like `{"amount": 450}`. The browser and the server talk in JSON.
Frontend / backend	Frontend is what runs in the browser. Backend is the Python code on the server.
Middleware / middle layer	The code between the screen and the database. Here: the Flask API.
Flask	A small Python tool for building web servers and APIs.
CRUD	Create, Read, Update, Delete: the four basic things you do with data.
Validation	Checking that data is allowed before saving it.
MongoDB	A database that stores data as documents, which look like JSON.

Word	Meaning
Collection / document	A collection is like a table. A document is like one row, written as JSON.
Compass	A desktop app to look at and edit a MongoDB database.
pymongo	The Python package that connects to MongoDB.
Pandas / DataFrame	A Python tool for working with table data. A DataFrame is a table in memory.
Matplotlib	A Python tool that draws charts.
SVG	A picture made of shapes and text, drawn sharply in the browser.
PNG	A normal picture file. Matplotlib saves charts as PNG.
CORS	A browser safety rule about which websites may call an API. The app allows only local addresses.
SPA (single-page app)	One web page whose content changes with JavaScript, without reloading.
Hash route	The part of the address after #, like `#expenses`, used to know which page to show.
Toast	A small message that appears for a few seconds, such as “Expense added”.
Skeleton	A grey placeholder shown while data is loading.
Fallback	A backup plan. If MongoDB is not available, the app uses JSON files.
mongomock	A fake MongoDB that lives in memory. Used only to run tests without a server.

18. Quick reference card

One last tip
When something looks wrong, ask three questions in order: **1)** Is the backend running and is the Storage line what I expect? **2)** What does the terminal say? **3)** What does the browser’s Network tab show for the failing request? These three answers solve most problems.

Thing	Value
Start the app	`cd backend` then `python app.py`
App address	`http://127.0.0.1:5000`
MongoDB address	`mongodb://localhost:27017`
Database / collections	`spendly` → `expenses`, `config`, `counters`, `meta`
Run the tests	`python tests.py`, `python tests_mongo.py`, `python tests_config.py`
Hard refresh the browser	Ctrl + Shift + R
Main backend file	`backend/app.py` (routes and errors)
Main frontend file	`frontend/js/app.js`
Validation rules	`expense_manager.py` (expenses) and `config_store.py` (settings, categories)
Calculations	`backend/analytics.py`

Thing	Value
Charts	`backend/charts.py` → `charts/` folder
Switch storage	`STORAGE=json` or `MONGODB_URI=...`